

Prelab Document for UART Interrupt-Based Echo with LED Control (Experiment 7.2)

Experiment Overview

This experiment implements part 7.2 of Experiment 7: Serial Asynchronous Communications, but using UART interrupts instead of polling. The ATmega328P connects to a PC terminal via UART (PD0 as RX input, PD1 as TX output). It receives a character from the PC. If the character is '1', it echoes back '*' followed by '1' and turns the LED on PC0 ON. Otherwise, it echoes back the received character and turns the LED OFF. All communication is non-blocking using interrupts: USART_RX_vect for receiving characters and USART_UDRE_vect for transmitting when multiple characters need to be sent. The baud rate is set to 19200 bps (not 9600) assuming a 16 MHz system clock. No functions are defined, and all logic is handled in main for initialization and ISRs for operation.

Concepts from course slides:

- From Class5_UART.pdf: Asynchronous serial communication (no clock shared, uses start/stop bits for framing), half/full-duplex (full-duplex here as RX and TX independent), baud rate calculation ($UBRR = f_{osc} / (16 * baud) - 1$), UART registers (UDR0 for data, UCSR0A for flags like RXC0/UDRE0, UCSR0B for enabling RX/TX/interrupts, UCSR0C for frame format: 8 data bits, 1 stop bit, no parity).
- From Class3_Interrupts.pdf: Interrupt function (breaks normal program flow on hardware event like data received), setup in ATmega328P (enable specific interrupts in UCSR0B like RXCIE0/UDRIE0, global enable with sei()), handling in AVR-C (use ISR(vector) macro, keep ISRs short).
- From Class1_IOports_v1 embedded.pdf: I/O port operation (DDRC sets direction: 1=output for LED), electrical characteristics (from datasheet: pins source/sink up to 40mA, but use resistor for LED), write to PORTC to set output high/low.

Initialization

1. Register: UCSR0A a. Value: 0b00000000 b. Configuration(s): i. Disable multi-processor mode (MPCM0=0), normal speed (U2X0=0), clear flags. c. Code: UCSR0A = 0;
2. Register: UCSR0B a. Value: 0b10011000 b. Configuration(s): i. Enable receiver (RXEN0=1), transmitter (TXEN0=1), RX complete interrupt (RXCIE0=1). UDRE interrupt

- (UDRIE0) enabled dynamically. c. Code: $UCSR0B = (1 \ll RXEN0) | (1 \ll TXEN0) | (1 \ll RXCIE0)$;
3. Register: UCSR0C a. Value: 0b00000110 b. Configuration(s): i. Asynchronous mode (UMSEL00/01=0), no parity (UPM00/01=0), 1 stop bit (USBS=0), 8 data bits (UCSZ00=1, UCSZ01=1, UCSZ02=0 in UCSR0B). c. Code: $UCSR0C = (1 \ll UCSZ01) | (1 \ll UCSZ00)$;
 4. Register: UBRR0 a. Value: 51 b. Configuration(s): i. Set baud rate to 19200 at 16 MHz: $UBRR = 16000000 / (16 * 19200) - 1 \approx 51$. c. Code: $UBRR0 = 51$;
 5. Register: DDRD a. Value: 0bxxxxxx1x (after operations) b. Configuration(s): i. Set PD1 as output for TX (though UART overrides when TXEN=1). ii. Set PD0 as input for RX. c. Code: $DDRD |= (1 \ll PD1)$; $DDRD \&= \sim(1 \ll PD0)$;
 6. Register: DDRC a. Value: 0bxxxxxxx1 (after |=) b. Configuration(s): i. Set PC0 as output for LED. c. Code: $DDRC |= (1 \ll PC0)$;
 7. Register: PORTC a. Value: Clear PC0 bit. b. Configuration(s): i. Default LED OFF (assuming active high: 0=OFF). c. Code: $PORTC \&= \sim(1 \ll PC0)$;

Table 1 Register Summary Table

Register Name	Function
UCSR0A	USART control and status A: speed mode, flags
UCSR0B	USART control and status B: enable RX/TX/interrupts
UCSR0C	USART control and status C: frame format
UBRR0	Baud rate setting
DDRD	Data direction for Port D: RX/TX pins
DDRC	Data direction for Port C: LED pin
PORTC	Port C output: control LED state

Infinite Loop

The infinite loop is empty as all operations are handled by interrupts. The MCU waits for RX interrupts from incoming data, processes it, and uses TX interrupts if needed for multi-character responses.

Table 2 Register Summary Table

Register	Bit	Purpose
-----------------	------------	----------------

(None in loop) N/A Interrupts handle all RX/TX/LED logic

Interrupt Service Routines

1. USART_RX_vect (RX Complete Interrupt):
 - Triggered when a full character is received (RXC0 flag set automatically).
 - Read received character from UDR0.
 - If '1', set LED ON, prepare '*' and '1' for TX.
 - Else, set LED OFF, prepare single character for TX.
 - Poll UDRE0 briefly to ensure TX buffer empty (short as baud is slow), then load first character to UDR0.
 - If second character pending, enable UDRE interrupt to send it next.
 - Concept: Allows non-blocking receive; MCU can do other tasks (though none here).
2. USART_UDRE_vect (UDR Empty Interrupt):
 - Triggered when TX buffer empty (UDRE0=1 after previous send).
 - Load second character to UDR0.
 - Clear pending indicator.
 - Disable UDRE interrupt (no more data).
 - Concept: Enables non-blocking transmit for multi-char responses without polling in main.

Table 3 Register Summary Table (ISRs)

Register	Bit	Purpose
UDR0	All	Read received data in RX ISR; write transmit data in both ISRs
UCSR0A	UDRE0	Check if TX buffer empty before writing in RX ISR
UCSR0B	UDRIE0	Enable/disable TX empty interrupt in both ISRs
PORTC	PC0	Update LED state in RX ISR

FLow chart

UART Interrupt-Driven Program Flow

